Interactive ERD schema: explore higher-ed tables across housing, dining, admissions, and registrar modules with live RBAC mapping

Computer
LTI 1.3 and FERPA audit monitor: build an automated compliance checker for student data access and learning tool integration
Computer
Show the pgTAP migration tests for tenant isolation
Draft the retention safeguard for FERPA disclosure records
Show the SQL schema for the housing and dining modules

I generated a housing/dining SQL reference containing 55 tables, a pgTAP baseline suite with 10 assertions, and review notes. Below are the key schema excerpts, isolation tests, and a FERPA retention safeguard.

The SQL is for a fresh reference database—not an upgrade over the earlier 129-table schema. It has not been executed against PostgreSQL, and the tests have not been run.

1. Housing and dining SQL

Included tables

Domain	Tables	Coverage
Housing	23	Inventory, applications, eligibility, roommate groups, selection, allocations, contracts, inspections, maintenance, deadlines, checkout, assessments, appeals
Dining	23	Plans, benefits, enrollment, entitlements, wallets, menus, transactions, disputes, adjustments, snapshots
Shared dependencies	9	Institution, people, campuses, terms, policies, documents, integrations, portal-facing charges and payment orders
Total	55	Complete for this housing/dining reference scope

The attached SQL contains all table definitions, composite foreign keys, indexes, constraints, version triggers, append-only protections, and default-deny RLS.

Its shared billing/payment tables are reduced portal-facing records, not a replacement for the earlier financial schema or an institutional general ledger.

Housing allocation schema

Reservations and confirmed occupancy share one allocation table. This lets the database prevent competing claims on the same bed.

```
CREATE TABLE semester.housing_allocations (
    tenant_id uuid NOT NULL,
    id uuid NOT NULL DEFAULT gen_random_uuid(),

    cycle_id uuid NOT NULL,
    bed_id uuid NOT NULL,
    person_id uuid NOT NULL,

    occupancy_range tstzrange NOT NULL,

    state text NOT NULL CHECK (
        state IN (
            'reserved',
            'offered',
            'contracted',
            'occupied',
            'completed',
            'cancelled',
            'expired'
        )
    ),

    expires_at timestamptz,
    request_key text NOT NULL,

    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),
    version bigint NOT NULL DEFAULT 1,

    PRIMARY KEY (tenant_id, id),
    UNIQUE (tenant_id, request_key),

    FOREIGN KEY (tenant_id, cycle_id)
        REFERENCES semester.housing_cycles (tenant_id, id),

    FOREIGN KEY (tenant_id, bed_id)
        REFERENCES semester.housing_beds (tenant_id, id),

    FOREIGN KEY (tenant_id, person_id)
        REFERENCES semester.people (tenant_id, id),

    CHECK (
        NOT isempty(occupancy_range)
        AND NOT lower_inf(occupancy_range)
        AND NOT upper_inf(occupancy_range)
        AND lower_inc(occupancy_range)
        AND NOT upper_inc(occupancy_range)
    ),

    CHECK (
```

```
        state <> 'reserved'
        OR expires_at IS NOT NULL
    )
);

ALTER TABLE semester.housing_allocations
ADD CONSTRAINT housing_bed_overlap
EXCLUDE USING gist (
    tenant_id WITH =,
    bed_id WITH =,
    occupancy_range WITH &&
)
WHERE (
    state IN ('reserved', 'offered', 'contracted', 'occupied')
);

ALTER TABLE semester.housing_allocations
ADD CONSTRAINT housing_person_overlap
EXCLUDE USING gist (
    tenant_id WITH =,
    person_id WITH =,
    occupancy_range WITH &&
)
WHERE (
    state IN ('reserved', 'offered', 'contracted', 'occupied')
);
```

The full migration creates `btree_gist` before these constraints.

Reservation expiration must explicitly change the state. Confirmation must check expiration inside the same locked transaction; a timestamp alone does not release the allocation.

Dining funding schema

A transaction line uses either a monetary wallet or a quantity entitlement—not both.

```
CREATE TABLE semester.dining_transaction_lines (
    tenant_id uuid NOT NULL,
    id uuid NOT NULL DEFAULT gen_random_uuid(),

    transaction_id uuid NOT NULL,
    wallet_id uuid,
    entitlement_period_id uuid,

    amount_minor bigint,
    quantity integer,

    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),
    version bigint NOT NULL DEFAULT 1,

    PRIMARY KEY (tenant_id, id),

    FOREIGN KEY (tenant_id, transaction_id)
```

```
        REFERENCES semester.dining_transactions (tenant_id, id),

    FOREIGN KEY (tenant_id, wallet_id)
        REFERENCES semester.campus_wallets (tenant_id, id),

    FOREIGN KEY (tenant_id, entitlement_period_id)
        REFERENCES semester.dining_entitlement_periods (tenant_id, id),

    CHECK (
        (
            wallet_id IS NOT NULL
            AND entitlement_period_id IS NULL
            AND amount_minor IS NOT NULL
            AND amount_minor > 0
            AND quantity IS NULL
        )
        OR
        (
            wallet_id IS NULL
            AND entitlement_period_id IS NOT NULL
            AND quantity IS NOT NULL
            AND quantity > 0
            AND amount_minor IS NULL
        )
    )
);
```

Append-only wallet entries

```
CREATE TABLE semester.campus_wallet_entries (
    tenant_id uuid NOT NULL,
    id uuid NOT NULL DEFAULT gen_random_uuid(),

    wallet_id uuid NOT NULL,
    transaction_line_id uuid,
    adjustment_request_id uuid,
    payment_order_id uuid,
    reverses_entry_id uuid,

    delta_minor bigint NOT NULL CHECK (delta_minor <> 0),
    entry_type text NOT NULL,
    idempotency_key text NOT NULL,
    posted_at timestamptz NOT NULL,

    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),
    version bigint NOT NULL DEFAULT 1,

    PRIMARY KEY (tenant_id, id),
    UNIQUE (tenant_id, idempotency_key),

    FOREIGN KEY (tenant_id, wallet_id)
        REFERENCES semester.campus_wallets (tenant_id, id),

    FOREIGN KEY (tenant_id, transaction_line_id)
```

```
        REFERENCES semester.dining_transaction_lines (tenant_id, id),

    FOREIGN KEY (tenant_id, adjustment_request_id)
        REFERENCES semester.dining_adjustment_requests (tenant_id, id),

    FOREIGN KEY (tenant_id, payment_order_id)
        REFERENCES semester.payment_orders (tenant_id, id),

    FOREIGN KEY (tenant_id, reverses_entry_id)
        REFERENCES semester.campus_wallet_entries (tenant_id, id)
);
```

The full SQL adds an update/delete rejection trigger. Corrections produce new reversing or adjustment entries.

Server commands must still verify account ownership, currency, valid benefits, matching-account reversals, payment confirmation, and concurrent spend authorization. The schema cannot safely prevent overdrafts with a cross-row `CHECK` constraint.

2. pgTAP isolation tests

Baseline test scope

The attached suite checks:

- RLS enabled and forced.
- Tenant columns included in tenant-owned foreign keys.
- Housing overlap constraint present.
- Runtime probe cannot bypass RLS.
- Default-deny visibility.
- Unauthorized inserts rejected.
- Cross-tenant person references rejected.

pgTAP's `throws_ok` can check SQLSTATE without depending on localized error text, and transaction-wrapped tests can roll back fixtures and test roles.[1]

Structural tests

```
BEGIN;

CREATE EXTENSION IF NOT EXISTS pgtap;
SELECT plan(3);

SELECT ok(
    NOT EXISTS (
        SELECT 1
        FROM pg_class c
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE n.nspname = 'semester'
```

```
          AND c.relkind IN ('r', 'p')
          AND NOT c.relrowsecurity
    ),
    'All Semester tables enable RLS'
);

SELECT ok(
    NOT EXISTS (
        SELECT 1
        FROM pg_class c
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE n.nspname = 'semester'
          AND c.relkind IN ('r', 'p')
          AND NOT c.relforcerowsecurity
    ),
    'All Semester tables force RLS'
);

SELECT ok(
    NOT EXISTS (
        SELECT 1
        FROM pg_constraint fk
        JOIN pg_class child ON child.oid = fk.conrelid
        JOIN pg_namespace n ON n.oid = child.relnamespace

        WHERE fk.contype = 'f'
          AND n.nspname = 'semester'

          AND EXISTS (
              SELECT 1
              FROM pg_attribute a
              WHERE a.attrelid = fk.confrelid
                AND a.attname = 'tenant_id'
                AND NOT a.attisdropped
          )

          AND NOT EXISTS (
              SELECT 1
              FROM unnest(fk.conkey, fk.confkey)
                   AS k(child_num, parent_num)

              JOIN pg_attribute ca
                ON ca.attrelid = fk.conrelid
               AND ca.attnum = k.child_num

              JOIN pg_attribute pa
                ON pa.attrelid = fk.confrelid
               AND pa.attnum = k.parent_num

              WHERE ca.attname = 'tenant_id'
                AND pa.attname = 'tenant_id'
          )
    ),
    'Tenant-owned foreign keys include tenant_id'
);
```

```
SELECT * FROM finish();
ROLLBACK;
```

Cross-tenant reference test

Run fixture creation as an isolated migration administrator capable of bypassing RLS.

```
BEGIN;

CREATE EXTENSION IF NOT EXISTS pgtap;
SELECT plan(1);

INSERT INTO semester.institutions (
    id, name, slug, default_timezone, status
)
VALUES
(
    '10000000-0000-0000-0000-000000000001',
    'Test University A',
    'isolation-a',
    'America/Chicago',
    'active'
),
(
    '20000000-0000-0000-0000-000000000002',
    'Test University B',
    'isolation-b',
    'America/Chicago',
    'active'
);

INSERT INTO semester.people (
    tenant_id, id, preferred_name, status
)
VALUES (
    '20000000-0000-0000-0000-000000000002',
    '30000000-0000-0000-0000-000000000003',
    'University B student',
    'active'
);

SELECT throws_ok(
    $sql$
    INSERT INTO semester.billing_charges (
        tenant_id,
        person_id,
        amount_minor,
        currency,
        status
    )
    VALUES (
        '10000000-0000-0000-0000-000000000001',
        '30000000-0000-0000-0000-000000000003',
        100,
        'USD',
```

```
        'pending'
    )
    $sql$,
    '23503',
    NULL,
    'University A cannot reference University B student'
);

SELECT * FROM finish();
ROLLBACK;
```

Tests still needed

The baseline schema denies all ordinary runtime access until reviewed policies are installed. That is not proof of a working authorization model.

Production test	Expected outcome
Student reads own housing assignment	Permitted
Student reads another student's assignment	Denied
University A housing staff reads B inventory/assignments	Denied
Dining officer views authorized A account	Permitted fields only
Faculty views dining purchase history	Denied by default
Advisor views unassigned student	Denied
Revoked delegation used on next request	Denied
Export attempts cross-tenant inclusion	Rejected or excludes unauthorized data
Two reservations target same bed	Only one succeeds
Duplicate dining event is processed twice	Only one financial/entitlement effect
Added broad policy changes access	Regression test fails

PostgreSQL combines permissive policies with `OR`; an accidentally broad policy can open access. Superusers and `BYPASSRLS` roles bypass RLS, so behavioral tests must use actual nonprivileged runtime roles. [2]

3. FERPA retention safeguard

Retention rule

Required FERPA request/disclosure records must remain for as long as the student's education records are maintained. Education records must not be destroyed while an inspection/review request is outstanding. Do not impose a universal fixed-year deletion rule on these records. [3]

This safeguard belongs in the full governance schema, not the reduced housing/dining reference.

Retention state

```
CREATE TABLE semester.disclosure_retention_reviews (
    tenant_id uuid NOT NULL,
    subject_person_id uuid NOT NULL,

    education_records_retained boolean NOT NULL DEFAULT true,
    institutional_hold boolean NOT NULL DEFAULT true,

    policy_version_id uuid NOT NULL,
    retirement_evidence_reference text,

    reviewed_by uuid NOT NULL,
    reviewed_at timestamptz NOT NULL,
    review_valid_until timestamptz NOT NULL,

    PRIMARY KEY (tenant_id, subject_person_id),

    FOREIGN KEY (tenant_id, subject_person_id)
        REFERENCES semester.people (tenant_id, id),

    FOREIGN KEY (tenant_id, policy_version_id)
        REFERENCES semester.policy_versions (tenant_id, id),

    FOREIGN KEY (tenant_id, reviewed_by)
        REFERENCES semester.people (tenant_id, id),

    CHECK (review_valid_until > reviewed_at),

    CHECK (
        education_records_retained
        OR retirement_evidence_reference IS NOT NULL
    )
);
```

Only an authorized records workflow may update this state. Deleting a Semester account does not establish that the institution no longer maintains education records.

Deletion guard

```
CREATE FUNCTION semester.guard_disclosure_retention()
RETURNS trigger
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = pg_catalog
AS $$
DECLARE
    r semester.disclosure_retention_reviews%ROWTYPE;
BEGIN
    SELECT *
    INTO r
    FROM semester.disclosure_retention_reviews
    WHERE tenant_id = OLD.tenant_id
      AND subject_person_id = OLD.subject_person_id
```

```
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'Disclosure deletion blocked: no approved retention review'
            USING ERRCODE = '55000';
    END IF;

    IF r.review_valid_until <= statement_timestamp() THEN
        RAISE EXCEPTION
            'Disclosure deletion blocked: retention review expired'
            USING ERRCODE = '55000';
    END IF;

    IF r.education_records_retained OR r.institutional_hold THEN
        RAISE EXCEPTION
            'Disclosure deletion blocked: records retained or hold active'
            USING ERRCODE = '55000';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM semester.student_data_requests q
        WHERE q.tenant_id = OLD.tenant_id
          AND q.subject_person_id = OLD.subject_person_id
          AND q.request_type = 'inspect'
          AND q.status NOT IN ('fulfilled', 'closed')
    ) THEN
        RAISE EXCEPTION
            'Disclosure deletion blocked: inspection request outstanding'
            USING ERRCODE = '55000';
    END IF;

    RETURN OLD;
END;
$$;

REVOKE ALL ON FUNCTION
    semester.guard_disclosure_retention()
FROM PUBLIC;

CREATE TRIGGER guard_disclosure_retention
BEFORE DELETE ON semester.ferpa_disclosure_records
FOR EACH ROW
EXECUTE FUNCTION semester.guard_disclosure_retention();
```

Mandatory safeguards around the trigger

- Apply default-deny RLS and reviewed grants to the retention table.
- Use a dedicated disposition role with visibility into every applicable request and hold.
- Do not allow ordinary users to set retirement flags.
- Audit approved disposition before removing records.
- Revoke `TRUNCATE`; row-level deletion triggers do not handle it.

- Coordinate request receipt, hold placement, and deletion using shared locking or an equivalent concurrency protocol.
- Block on missing, stale, or incomplete retirement evidence.
- Do not treat a hidden request—filtered out by RLS—as proof that no request exists.
- Maintain separate policies for operational logs, required disclosure records, files, backups, and externally held records.

PostgreSQL explicitly excludes `TRUNCATE` from row-security enforcement. This is why grants and operational controls are necessary alongside triggers. [2]

Retention test matrix

Condition	Expected
No retention review	Reject deletion
Education records remain	Reject deletion
Institutional/legal hold active	Reject deletion
Review expired	Reject deletion
Outstanding inspection request	Reject deletion
Request becomes outstanding during disposition	Shared locking prevents unsafe deletion
Approved retirement, valid review, no requests/holds	Eligible for audited disposition
Ordinary application role attempts deletion	Permission denied
Ordinary role attempts `TRUNCATE`	Permission denied

4. ERD and compliance monitor

Interactive ERD

Expose schema metadata and control evidence, not unrestricted production student rows.

For each selected table, show:

- Columns, tenant key, foreign keys, constraints, and indexes.
- Source-of-truth mode and synchronization freshness.
- Roles, scopes, relationships, and permitted field categories.
- Server commands that may mutate it.
- Audit/disclosure requirements.
- Migration and test evidence.
- Implemented, untested, or specification-only status.

Automated checker

Check	Evidence	Output
Tenant isolation	Database catalogs and behavioral tests	Missing tenant FK, RLS, or failed boundary
Role access	Assignments, scopes, policy version	Unauthorized or expired grant
Consent	Applicable basis and validated consent evidence	Missing/invalid consent only where required
Disclosure recordkeeping	Actual releases plus institutional classification	Required record missing or incomplete
Retention	Reviews, record status, outstanding requests, holds	Disposition blocked or review needed
LTI integration	Trusted registration/deployment and validation result	Unknown deployment, replay, invalid token
Sync reliability	Jobs, cursors, source timestamps, reconciliation	Stale or conflicting authoritative data
Student requests	Receipt, applicable deadline, owner, progress	Deadline risk or preservation failure

Keep access auditing separate from legally required disclosure recordkeeping. FERPA's recordkeeping provisions have exceptions; the checker must use approved institutional classification rather than assuming every access requires consent or a disclosure entry.[3]

The monitor should display specific control failures and supporting evidence—not a blanket "FERPA compliant" badge.

⁂

1. https://pgtap.org/documentation.html
2. https://www.postgresql.org/docs/current/ddl-rowsecurity.html
3. https://studentprivacy.ed.gov/ferpa